

第三章 程序的转换及机器级表示

一、名词解释

1. 机器语言程序：一种由二进制代码组成的程序，直接由计算机硬件执行的指令集合。
2. 汇编指令：一种由助记符表示的机器指令，与机器语言一一对应，可被汇编器转换成对应的机器语言指令。
3. 汇编语言程序：使用汇编语言编写的程序，以助记符形式表示的指令序列，需要经过汇编器转换为机器语言程序。
4. 汇编助记符：在汇编语言中使用的符号，代表对应的机器指令。
5. 汇编程序：将汇编语言程序转换成机器语言程序的程序，也称为汇编器。
6. 反汇编程序：将机器语言程序转换回汇编语言程序的程序。
7. 机器级代码：直接在计算机硬件上执行的指令，通常是由汇编器将汇编语言转换而来的。
8. CISC：复杂指令集计算机，指一种计算机架构，其指令集包含了多种复杂的指令。
9. RISC：精简指令集计算机，指一种计算机架构，其指令集相对简单，每条指令的执行时间相对较短。
10. 通用寄存器：用于存储临时数据和地址的寄存器，通常用于存储计算过程中的中间结果。
11. 变址寄存器：用于存储地址计算中的偏移量的寄存器，通常用于实现数组、结构体等数据结构的访问。
12. 基址寄存器：用于存储内存地址基址的寄存器，通常与变址寄存器一起使用，用于计算实际内存地址。
13. 栈指针寄存器：用于存储栈顶地址的寄存器，用于实现函数调用和局部变量存储等操作。
14. 指令指针寄存器：用于存储当前正在执行指令的地址的寄存器，通常指向下一条要执行的指令。
15. 标志寄存器：用于存储程序运行状态标志的寄存器，例如零标志、进位标志等。
16. 条件标志(条件码)：标志寄存器中的一部分，用于表示上一条指令的执行结果，通常用于控制条件跳转指令的执行。
17. 寻址方式：指定操作数的寻址方式，例如立即寻址、寄存器寻址、相对寻址等。
18. 立即寻址：直接将操作数的值嵌入到指令中。
19. 寄存器寻址：使用寄存器存储操作数的地址。
20. 相对寻址：使用相对于某个基址的偏移量来寻址。
21. 存储器操作数：指令中的操作数直接引用存储器中的数据。
22. 实地址模式：CPU 直接使用物理地址访问内存的模式。
23. 保护模式：一种 CPU 工作模式，操作系统可在其中对内存和其他资源进行保护和管理。
24. 有效地址：执行指令时计算出的操作数在内存中的真实地址。
25. 比例变址：根据一个基址寄存器和一个比例系数来计算操作数的地址。

26. 非比例变址：仅根据一个基址寄存器来计算操作数的地址。
27. 比例系数(比例因子)：用于比例变址计算的一个数值。
28. MMX 指令：Intel 处理器中用于多媒体处理的指令集。
29. SSE 指令集：Intel 处理器中的一种扩展指令集，用于实现单指令多数据（SIMD）操作。
30. SIMD：单指令多数据，一种并行计算的技术，能够同时处理多个数据。
31. 多媒体扩展通用寄存器：用于 SIMD 指令集操作的寄存器。
32. 栈(Stack)：一种后进先出（LIFO）的数据结构，用于临时存储函数调用的上下文和局部变量。
33. 调用者保存寄存器：在函数调用过程中，由调用者负责保存和恢复的寄存器。
34. 被调用者保存寄存器：在函数调用过程中，由被调用者负责保存和恢复的寄存器。
35. 帧指针寄存器：用于指向当前函数调用的栈帧的寄存器。
36. 当前栈帧：当前函数调用的栈帧，存储了函数的参数、局部变量和返回地址等信息。
37. 按值传递参数：将参数的值传递给函数。
38. 按地址传递参数：将参数的地址传递给函数，函数可以直接访问参数的值。
39. 嵌套调用：在一个函数内部调用另一个函数的过程。
40. 递归调用：函数直接或间接地调用自身的过程。

二、简单题

(1) 一条机器指令通常由哪些字段组成?各字段的含义分别是什么?

解答：一条机器指令通常由以下字段组成：

- 操作码 (Opcode)：指示该指令的操作类型，如加法、乘法等。
- 操作数 (Operand)：指令要操作的数据或数据的存储地址。
- 寻址方式 (Addressing Mode)：指定如何获取操作数的地址。
- 操作数长度 (Operand Length)：指定操作数的长度，通常以位或字节为单位。

(2) 将一个高级语言源程序转换成计算机能直接执行的机器代码通常需要哪几个步骤?

解析：将高级语言源程序转换成计算机能直接执行的机器代码通常需要以下步骤：

- 词法分析 (Lexical Analysis)：将源代码分割成词法单元。
- 语法分析 (Syntax Analysis)：将词法单元组合成语法结构。
- 语义分析 (Semantic Analysis)：检查语法结构是否符合语言规范。
- 代码生成 (Code Generation)：将语法结构翻译成目标机器的机器代码。
- 代码优化 (Code Optimization)：对生成的机器代码进行优化，以提高执行效率。

(3) IA-32 中的逻辑运算指令如何生成条件标志?移位指令可能会改变哪些条件标志?

解析：在 IA-32 中，逻辑运算指令（如 AND、OR、XOR）会根据操作数的结果生成条件标志（如零标志 ZF、进位标志 CF 等）。移位指令（如 SHL、SHR）可能会改变零标志 ZF、进位标志 CF 和溢出标志 OF。

(4) 执行条件跳转指令时所用到的条件标志信息从何而来?请举例说明。

解析：执行条件跳转指令时所用到的条件标志信息来自于上一条影响条件标志的指令。例如，执行 JZ（如果零标志为 1 则跳转）指令时，需要根据上一条指令的执行结果来确定是否跳转。

(5)无条件跳转指令和调用指令的相同点和不同点是什么？

解析：无条件跳转指令和调用指令的相同点是它们都可以改变程序执行的流程，使程序跳转到指定的地址执行。不同点在于无条件跳转指令只是简单地改变程序计数器的值，而调用指令还会将当前的程序状态（如返回地址）保存到栈中，以便后续返回。

(6)按值传递参数和按地址传递参数两种方式有哪些不同点？

解析：按值传递参数和按地址传递参数的不同点在于：

- 按值传递参数会将参数的值复制给函数的形式参数，因此函数内部对形式参数的修改不会影响实际参数的值；
- 按地址传递参数则会将参数的地址传递给函数，函数内部对形式参数的修改会影响实际参数的值。

(7)为什么在递归深度较深时递归调用的时间开销和空间开销都会较大？

解析：当递归深度较深时，递归调用的时间开销和空间开销都会较大是因为每次递归调用都会占用额外的栈空间，并且频繁的函数调用和返回会增加时间开销。

(8)为什么数据在存储器中最好按地址对齐方式存放？

解析：数据在存储器中最好按地址对齐方式存放是因为许多计算机体系结构对数据的访问有地址对齐的要求。如果数据没有按照正确的地址对齐方式存放，可能会导致额外的内存访问，降低访问效率。

3.对于以下 AT&T 格式汇编指令，根据操作数的长度确定对应指令助记符中的长度后缀，并说明每个操作数的寻址方式。

- (1) `mov 8(%ebp, %ebx, 4), %ax`
- (2) `mov %al, 12(%ebp)`
- (3) `add (, %ebx, 4), %ebx`
- (4) `or (%ebx), %dh`
- (5) `push $0xF8`
- (6) `mov $0xFFFF0, %eax`
- (7) `test %cx, %cx`
- (8) `lea 8(%ebx, %esi), %eax`

解析：

对于给出的 AT&T 格式汇编指令，我们将根据操作数的长度确定对应指令助记符中的长度后缀，并说明每个操作数的寻址方式。

- (1) ``mov 8(%ebp, %ebx, 4), %ax``
 - 指令助记符：``mov``
 - 操作数长度后缀：``w``（表示 word，16 位）
 - 操作数 1：``8(%ebp, %ebx, 4)``，寻址方式为 ``[base + index * scale + displacement]``。这里 ``8(%ebp, %ebx, 4)`` 表示从内存地址 ``[ebp + ebx * 4 + 8]`` 中读取数据。
 - 操作数 2：``%ax``，寻址方式为寄存器直接。
- (2) ``mov %al, 12(%ebp)``
 - 指令助记符：``mov``

- 操作数长度后缀: ``b`` (表示 byte, 8 位)
- 操作数 1: ``%al``, 寻址方式为寄存器直接。
- 操作数 2: ``12(%ebp)``, 寻址方式为 ``[base + displacement]``。这里 ``12(%ebp)`` 表示从内存地址 ``[ebp + 12]`` 中读取数据。

(3) ``add(, %ebx, 4), %ebx``

- 指令助记符: ``add``
- 操作数长度后缀: 根据默认寄存器长度确定, 这里 ``%ebx`` 是 32 位寄存器, 因此默认为 ``l`` (long, 32 位)
- 操作数 1: ``(, %ebx, 4)``, 寻址方式为 ``[index * scale]``。这里 ``(, %ebx, 4)`` 表示从内存地址 ``[ebx * 4]`` 中读取数据。

- 操作数 2: ``%ebx``, 寻址方式为寄存器直接。

(4) ``or(%ebx), %dh``

- 指令助记符: ``or``
- 操作数长度后缀: ``b`` (表示 byte, 8 位)
- 操作数 1: ``(%ebx)``, 寻址方式为 ``[ebx]``, 从内存地址 ``[ebx]`` 中读取数据。
- 操作数 2: ``%dh``, 寻址方式为寄存器直接。

(5) ``push $0xF8``

- 指令助记符: ``push``
- 操作数长度后缀: N/A, 立即数直接推送到栈中, 大小与操作数大小无关。
- 操作数: ``$0xF8``, 即立即数 ``0xF8``。

(6) ``mov $0xFFFF0, %eax``

- 指令助记符: ``mov``
- 操作数长度后缀: 默认为 ``l`` (long, 32 位)
- 操作数 1: ``$0xFFFF0``, 即立即数 ``0xFFFF0``。
- 操作数 2: ``%eax``, 寻址方式为寄存器直接。

(7) ``test %cx, %cx``

- 指令助记符: ``test``
- 操作数长度后缀: 根据 ``%cx`` 的寄存器大小确定, 默认为 ``w`` (word, 16 位)
- 操作数 1: ``%cx``, 寄存器直接。
- 操作数 2: ``%cx``, 寄存器直接。

(8) ``lea 8(%ebx, %esi), %eax``

- 指令助记符: ``lea`` (Load Effective Address)
- 操作数长度后缀: 默认为 ``l`` (long, 32 位)
- 操作数 1: ``8(%ebx, %esi)``, 寻址方式为 ``[ebx + esi + 8]``, 将有效地址加载到 ``%eax`` 寄存器中。
- 操作数 2: ``%eax``, 寄存器直接。

4. 使用汇编器处理以下各行 AT&T 格式代码时都会产生错误, 请说明每一行存在什么错误。

(1) `movl 0xFF, ( %eax)`

```

(2) movb %ax, 12(%ebp)
(3) addl %ecx, $0xF0
(4) orw $0xFFFF0, (%ebx)
(5) addb $0xF8, (%dl)
(6) movl %bx, %eax
(7) andl %esi, %esx
(8) movw 8(%ebp, , 4), %ax

```

解析：

当使用汇编器处理以下 AT&T 格式代码时会产生错误，以下是每一行存在的错误：

(1) ``movl 0xFF, (%eax)``

错误：在内存寻址时，地址应该是有效的内存地址，但这里 ``(%eax)`` 之间多了一个空格，应该是 ``(%eax)`` 才是正确的寻址方式。另外，立即数 ``0xFF`` 的长度是一个字节，但这里使用的是 ``movl`` 指令，该指令表示操作数是一个 32 位长的值，因此需要使用 ``movb`` 指令。

(2) ``movb %ax, 12(%ebp)``

错误：在将 ``%ax`` 寄存器内容移动到内存时，``%ax`` 是一个 16 位寄存器，但是目的操作数 ``12(%ebp)`` 表示的是一个字节。这意味着 16 位的 ``%ax`` 中的值不能完全放入一个字节的目的地位置中，应该使用 ``movw`` 指令来将 16 位的数据移动到内存。

(3) ``addl %ecx, $0xF0``

错误：在 ``addl`` 指令中，第一个操作数应该是目的操作数，但是这里 ``$0xF0`` 是一个立即数，应该放在第二个操作数的位置上。正确的语法应该是 ``addl $0xF0, %ecx``。

(4) ``orw $0xFFFF0, (%ebx)``

错误：立即数 ``$0xFFFF0`` 超出了 16 位寄存器 ``%ebx`` 可以表示的范围（16 位）。应该使用适当大小的立即数来避免溢出。

(5) ``addb $0xF8, (%dl)``

错误：``%dl`` 是一个 8 位的寄存器，而 ``addb`` 指令是用来对 8 位操作数进行加法运算的。因此，这里使用 ``%dl`` 作为目的操作数是不正确的。应该选择一个 32 位或者 16 位的寄存器作为目的操作数。

(6) ``movl %bx, %eax``

错误：``%bx`` 是一个 16 位寄存器，而 ``%eax`` 是一个 32 位寄存器，它们的大小不匹配。应该使用 ``movw`` 指令来移动 ``%bx`` 的值到 ``%eax``，或者使用 ``movzwl`` 来将 ``%bx`` 的值扩展为 32 位。

(7) ``andl %esi, %esx``

错误：这里 ``%esx`` 应该是 ``%esi``，因为 ``%esi`` 是一个 32 位寄存器，而 ``%esx`` 似乎是一个错误的操作数。

(8) ``movw 8(%ebp, , 4), %ax``

错误：在计算内存地址时，括号中的表达式中缺少一个索引寄存器。应该是 ``movw 8(%ebp, %index, 4), %ax``，其中 ``%index`` 是一个索引寄存器。

5. 假设变量 `x` 和 `ptr` 的类型声明如下：

```
src_type x;
dst_type* ptr;
```

这里，src_type 和 dst_type 是用 typedef 声明的数据类型。有以下一个 C 语言赋值语句：

```
*ptr=( dst_type) x;
```

若 x 存储在寄存器 EAX 或 AX 或 AL 中，ptr 存储在寄存器 EDX 中，则对于表 3.12 中给出的 src_type 和 dst_type 的类型组合，写出实现上述赋值语句的机器级代码。要求用 AT&T 格式表示机器级代码。

对于给出的不同 src_type 和 dst_type 的组合，下面是实现赋值语句 `\*ptr = (dst\_type)x;` 的机器级代码，使用 AT&T 格式表示：

表 3.12 题 5 用表

src_type	dst_type	机器级表示
char	int	
int	char	
int	unsigned	
short	int	
unsigned char	unsigned	
char	unsigned	
int	int	

(1) `src\_type(char)`、`dst\_type(int)` 的机器级表示为：

```
movsbl %al, %eax # 将 x 的值从 AL 扩展为 32 位有符号整数，并存储到 EAX
movl %eax, (%edx) # 将 EAX 中的值存储到 ptr 所指向的内存地址中
```

(2) `src\_type(int)`、`dst\_type(char)` 的机器级表示为：

```
movb %al, (%edx) # 将 x 的低字节存储到 ptr 所指向的内存地址中
```

(3) `src\_type(int)`、`dst\_type(unsigned)` 的机器级表示为：

```
movl %eax, (%edx) # 将 x 的值存储到 ptr 所指向的内存地址中
```

(4) `src\_type(short)`、`dst\_type(int)` 的机器级表示为：

```
movswl %ax, %eax # 将 x 的值从 AX 扩展为 32 位有符号整数，并存储到 EAX
movl %eax, (%edx) # 将 EAX 中的值存储到 ptr 所指向的内存地址中
```

(5) `src\_type(unsigned char)`、`dst\_type(unsigned)` 的机器级表示为：

```
movzbl %al, %eax # 将 x 的值从 AL 扩展为 32 位无符号整数，并存储到 EAX
movl %eax, (%edx) # 将 EAX 中的值存储到 ptr 所指向的内存地址中
```

(6) `src\_type(char)`、`dst\_type(unsigned)` 的机器级表示为：

```
movsbl %al, %eax # 将 x 的值从 AL 扩展为 32 位有符号整数，并存储到 EAX
movl %eax, (%edx) # 将 EAX 中的值存储到 ptr 所指向的内存地址中
```

(7) `src\_type(int)`、`dst\_type(int)` 的机器级表示为：

```
movl %eax, (%edx) # 将 x 的值存储到 ptr 所指向的内存地址中
```

6. 假设某个 C 语言函数 func 的原型声明如下：

```
void func( int * xptr, int * yptr, int * zptr);
```

函数 func 的过程体对应的机器级代码用 AT&T 汇编形式表示如下：

```
(1) Movl 8( %ebp), %eax
(2) Movl 12(%ebp), %ebx
(3) movl 16( %ebp), %ecx
(4) Movl (%ebx), %edx
(5) Movl ( %ecx), %esi
(6) Movl ( %eax), %edi
(7) Movl %edi, ( %ebx)
(8) Movl %edx, ( %ecx)
(9) Movl %esi, ( %eax)
```

回答下列问题或完成下列任务。

(1) 在过程体开始时三个入口参数对应实参所存放的存储单元地址是什么(提示:当前栈帧底部由帧指针寄存器 EBP 指示)?

(2) 根据上述机器级代码写出函数 func 的 C 语言代码。

解析:

(1) 在过程体开始时, 三个入口参数对应实参所存放的存储单元地址如下:

- `xptr` 的地址存放在 `8(%ebp)` 中
- `yptr` 的地址存放在 `12(%ebp)` 中
- `zptr` 的地址存放在 `16(%ebp)` 中

这是因为在函数调用过程中, 参数是按顺序被压入栈中, 而函数的帧指针 `%ebp` 指向当前栈帧的底部, 所以在栈上偏移量为 `8(%ebp)`、`12(%ebp)`、`16(%ebp)` 的位置分别存储着相应参数的地址。

(2) 根据上述机器级代码, 函数 `func` 的 C 语言代码可以写成:

```
void func(int *xptr, int *yptr, int *zptr) {
    int x = *yptr;
    int y = *zptr;
    int z = *xptr;

    *xptr = y;
    *yptr = z;
    *zptr = x;
}
```

这段 C 代码实现了与给定机器级代码相对应的功能。

9. 假设函数 operate 的部分 C 代码如下:

```
1 int operate(int x, int y, int z, int k){
2 int v=____;
3 return v;
4 }
```

以下汇编代码用来实现第 2 行语句的功能，请写出每条汇编指令的注释，并根据以下汇编代码，填写 operate 函数缺失的部分。

```
(1) movl 12(%ebp), %ecx
(2) sall $8, %ecx
(3) movl 8(%ebp), %eax
(4) movl 20(%ebp), %edx
(5) imull %edx, %eax
(6) movl 16(%ebp), %edx
(7) andl $65520, %edx
(8) addl %ecx, %edx
(9) subl %edx, %eax
```

解析：

根据给出的汇编代码，我们可以添加注释，并填写 `operate` 函数缺失的部分：

```
1 int operate(int x, int y, int z, int k){
2     int v=____; // v 的初始值
3     return v;
4 }
```

汇编指令的注释如下：

1. `movl 12(%ebp), %ecx`：将参数 `z` 的值加载到 `%ecx` 寄存器中。
2. `sall \$8, %ecx`：将 `%ecx` 寄存器中的值左移 8 位（相当于乘以 256）。
3. `movl 8(%ebp), %eax`：将参数 `x` 的值加载到 `%eax` 寄存器中。
4. `movl 20(%ebp), %edx`：将参数 `k` 的值加载到 `%edx` 寄存器中。
5. `imull %edx, %eax`：将 `%edx` 寄存器中的值与 `%eax` 寄存器中的值相乘，并将结果存储到 `%eax` 中。
6. `movl 16(%ebp), %edx`：将参数 `y` 的值加载到 `%edx` 寄存器中。
7. `andl \$65520, %edx`：将 `%edx` 寄存器中的值与 65520 (0xFF00) 进行按位与运算。
8. `addl %ecx, %edx`：将 `%ecx` 寄存器中的值加到 `%edx` 寄存器中。
9. `subl %edx, %eax`：将 `%edx` 寄存器中的值从 `%eax` 寄存器中减去，并将结果存储到 `%eax` 中。

根据这些注释，我们可以填写缺失的部分，即变量 `v` 的初始化：

```
int v = (z << 8) + x * k - (y & 0xFF00);
```

10. 假设函数 product 的 C 语言代码如下，其中 num_type 是用 typedef 声明的数据类型。

```
(1) void product( num_type* d, unsigned x, num_type y){
(2)     *d = x*y;
(3) }
```

函数 product 的过程体对应的主要汇编代码如下：

- (1) `Movl 12(%ebp), %eax`
- (2) `Movl 20(%ebp), %ecx`
- (3) `imull %eax, %ecx`
- (4) `Mull 16(%ebp)`
- (5) `Leal (%ecx, %edx), %edx`
- (6) `Movl 8(%ebp), %ecx`
- (7) `Movl %eax, (%ecx)`
- (8) `Movl %edx, 4(%ecx)`

请给出上述每条汇编指令的注释，并说明 `num_type` 是什么类型。

解析：

下面是给出的汇编代码的注释以及关于 `num_type` 类型的说明：

1. `Movl 12(%ebp), %eax` // 将参数 `y` 的值加载到 `%eax` 寄存器中
2. `Movl 20(%ebp), %ecx` // 将参数 `x` 的值加载到 `%ecx` 寄存器中
3. `imull %eax, %ecx` // 将 `%ecx` 寄存器中的值与 `%eax` 寄存器中的值相乘，结果存储在 `%ecx` 中
4. `Mull 16(%ebp)` // 将参数 `d` 的地址加载到 `%edx: %eax` 寄存器中，同时将 `%ecx` 寄存器中的值与参数 `d` 所指向的内存地址处的值相乘，结果存储在 `%edx: %eax` 中
5. `Leal (%ecx, %edx), %edx` // 计算 `%ecx` 寄存器中的值加上 `%edx` 寄存器中的值的地址，结果存储在 `%edx` 寄存器中
6. `Movl 8(%ebp), %ecx` // 将参数 `d` 的地址加载到 `%ecx` 寄存器中
7. `Movl %eax, (%ecx)` // 将 `%eax` 寄存器中的值存储到 `%ecx` 寄存器中指向的内存地址中
8. `Movl %edx, 4(%ecx)` // 将 `%edx` 寄存器中的值存储到 `%ecx` 寄存器中指向的内存地址加上 4 字节处

根据汇编代码，`num_type` 是一个指向某种数据类型的指针。